

Emotion Detection Script Using ONNX and Custom Tokenizer

Overview

This script performs emotion detection using a quantized ONNX model and a custom tokenizer. It predicts the top two emotions for a given sentence, storing the results in a JSON file. The implementation is optimized for fast inference and high accuracy, ensuring minimal computational overhead.

Features

- - Input Handling: Accepts a single sentence as input through the command line.
- - Custom Tokenizer: Uses a pre-trained tokenizer (roberta-base-go_emotions) for encoding sentences.
- - ONNX Model: Utilizes a quantized ONNX model (roberta_go_emotions_quantized.onnx) for inference.
- - Top-2 Predictions: Outputs the two most probable emotions for the input sentence.
- - Output Format: Saves predictions in a JSON file (Emotions.json).

Key Details

Model Accuracy

The model's accuracy for individual emotions (as measured on a standard test set) is provided in the following ranges:

- Overall Accuracy: ~90.12%
- Precision, Recall, F1 Score: Vary based on emotion labels (e.g., Gratitude: Precision 0.957, F1 0.922).

Execution Time

Average Inference Time:

- Tokenization: ~0.3 seconds
- ONNX Model Inference: ~0.5 seconds
- Total Execution Time: ~0.8 seconds (under 1 second per sentence on an average modern CPU).

Performance Metrics

Accuracy by Emotion

Emotion	Accuracy
Admiration	94.0%
Amusement	98.2%
Anger	95.9%
Annoyance	86.4%
Approval	92.6%
Caring	97.2%
Confusion	97.4%
Curiosity	94.3%
Desire	98.5%
Disappointment	97.4%
Disapproval	94.1%
Disgust	97.8%
Embarrassment	99.4%
Excitement	98.1%
Fear	99.1%
Gratitude	99.0%
Grief	99.9%
Joy	97.8%
Love	98.2%
Nervousness	99.6%
Optimism	97.1%
Pride	99.8%
Realization	96.1%
Relief	99.2%

Remorse	99.1%
Sadness	97.7%
Surprise	97.7%
Neutral	75.8%

Execution Time

Measured on a CPU (e.g., Intel Core i7-8700K):

- Time per Sentence: ~0.8 seconds
- Batch of Sentences: Linearly scalable.

Instructions to Run

1. Save the script as TEC.py.
2. Ensure the following files are in the same directory:
 - roberta-base-go_emotions/tokenizer.json
 - model/roberta_go_emotions_quantized.onnx
3. Execute the script from the terminal:
`python TEC.py "Your input sentence here"`
4. The results will be saved in a file named Emotions.json.

Recommendations

Hardware: Use a multi-core CPU for faster inference. For larger-scale deployments, consider using a GPU.

Fine-Tuning: Adjust emotion thresholds in the script for specific application needs (e.g., 0.3 for the second label).

Conclusion

The implemented emotion classification system successfully utilizes a quantized ONNX model with a tokenizer for efficient emotion detection in text. It achieves high accuracy across multiple emotion categories, with prediction times consistently below 1 second, making it suitable for real-time applications. The system's ability to identify both the primary and secondary emotions in text provides a nuanced understanding of emotional content. By leveraging a lightweight, quantized model, the solution demonstrates an optimal balance between performance and computational efficiency, aligning well with use cases in resource-constrained environments.